

Análisis descriptivo de variables cualitativas

Issue #38 — MA2003B, equipo 7

Tabla de contenidos

0. Carga y construcción de las categóricas	1
periodo	2
tipo_dia y festivo	2
llovio	3
Temporada en el diario	3
1. Distribución de frecuencia	3
2. estacion cruzada con la cobertura temporal	6
3. Mediana en escala ordinal	6
4. Visualización	7
5. Criterios de aceptación	10

Issue #38. Contraparte cualitativa del análisis descriptivo. Opera sobre `sima_limpio_horario.csv` y `sima_limpio_diario.csv`, las dos salidas de `src/limpieza.py` (#13), **no** sobre `sima_transformado_horario.csv`, las variables cuantitativas de ese archivo están en escala Box-Cox estandarizada y las indicadoras de excedencia no viven ahí.

Las figuras se escriben en `reports/figuras/` y las cifras alimentan `reports/secciones/eda_cualitativas.tex`.

0. Carga y construcción de las categóricas

Tres variables en alcance no venían construidas y se generan aquí.

```
import pandas as pd, numpy as np, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from pathlib import Path

RAIZ = Path.cwd().parent
PROC = RAIZ / "data" / "processed"
FIGS = RAIZ / "reports" / "figuras"
FIGS.mkdir(parents=True, exist_ok=True)

H = pd.read_csv(PROC / "sima_limpio_horario.csv", parse_dates=["fecha"])
D = pd.read_csv(PROC / "sima_limpio_diario.csv", parse_dates=["fecha"])
```

```
print(f"horario: {H.shape}    diario: {D.shape}")
```

```
horario: (574343, 31)    diario: (23931, 9)
```

periodo

Binning de la hora en las cuatro fases del ciclo fotoquímico diario. Los cortes se heredan de `archive/02_transformacion_datos.qmd` y no se modifican, para que la variable sea comparable con lo ya reportado: bloques de seis horas, madrugada 00–05, mañana 06–11, tarde 12–17, noche 18–23.

```
ORDEN_PERIODO = ["madrugada", "mañana", "tarde", "noche"]
H["periodo"] = pd.cut(H["hora"], bins=[-1, 5, 11, 17, 23], labels=ORDEN_PERIODO)
```

tipo_dia y festivo

`fin_de_semana` agrupa sábado y domingo pese a que su actividad vehicular difiere, y no separa los días de asueto. Se construye un `tipo_dia` de cuatro niveles: días de descanso obligatorio de la LFT art. 74, jueves y viernes santo, y el periodo 24–31 de diciembre, que tienen patrón de tráfico anómalo y contaminarían cualquier contraste semanal si quedaran dentro de `laborable`.

```
PASCUA = {2021: "2021-04-04", 2022: "2022-04-17", 2023: "2023-04-09",
          2024: "2024-03-31", 2025: "2025-04-20"}

def calendario_festivos(anios=range(2021, 2026)) -> set:
    fest = set()
    for y in anios:
        fest |= {f"{y}-01-01", f"{y}-05-01", f"{y}-09-16", f"{y}-12-25"}
        for mes, n in [(2, 1), (3, 3), (11, 3)]:      # 1er lun feb, 3er lun mar y
            ↪ nov
            dias = pd.date_range(f"{y}-{mes:02d}-01", periods=31, freq="D")
            lunes = [d for d in dias if d.month == mes and d.weekday() == 0]
            fest.add(str(lunes[n - 1].date()))
            p = pd.Timestamp(PASCUA[y])
            fest |= {str((p - pd.Timedelta(days=k)).date()) for k in (3, 2)}
            fest |= {str(d.date()) for d in pd.date_range(f"{y}-12-24", f"{y}-12-31")}
        fest.add("2024-10-01")      # transmisión del Poder Ejecutivo federal
    return {pd.Timestamp(f) for f in fest}

FESTIVOS = calendario_festivos()

def marcar_calendario(df: pd.DataFrame) -> pd.DataFrame:
    dw = df["fecha"].dt.dayofweek
    df["festivo"] = df["fecha"].dt.normalize().isin(FESTIVOS).astype(int)
    df["tipo_dia"] = np.select([dw == 5, dw == 6], ["sabado", "domingo"],
    ↪ "laborable")
```

```
df.loc[df.festivo == 1, "tipo_dia"] = "festivo"
df["tipo_dia"] = pd.Categorical(
    df.tipo_dia, categories=["laborable", "sabado", "domingo", "festivo"])
return df
```

```
H = marcar_calendario(H); D = marcar_calendario(D)
```

llovio

Dicotomización de RAINF. A diferencia de la versión de #14, **los faltantes se conservan como faltantes**: asignarles 0 los reclasifica como “no llovió” e infla esa clase con horas en las que simplemente no se midió.

```
H["llovio"] = np.where(H.RAINF.isna(), np.nan, (H.RAINF > 0).astype(float))
print("llovio=1:", int((H.llovio == 1).sum()),
      "| sin dato:", int(H.llovio.isna().sum()))
```

```
llovio=1: 7988 | sin dato: 17251
```

Temporada en el diario

```
MES_TEMP = {12:"invierno",1:"invierno",2:"invierno",3:"primavera",4:"primavera",
             5:"primavera",6:"verano",7:"verano",8:"verano",
             9:"otoño",10:"otoño",11:"otoño"}
D["temporada"] = D.fecha.dt.month.map(MES_TEMP)
D["fin_de_semana"] = D.fecha.dt.dayofweek.isin([5, 6]).astype(int)
```

1. Distribución de frecuencia

periodo y llovio son horarias; las indicadores de excedencia derivan del MDA8 y son diarias. Si se tabulan juntas, cada valor diario se repite 24 veces y el porcentaje de excedencia deja de coincidir con el publicado en #14.

```
def frecuencia(s: pd.Series, nombre: str) -> pd.DataFrame:
    vc = s.value_counts(dropna=False)
    t = pd.DataFrame({"n": vc})
    t["pct_total"] = (t.n / t.n.sum() * 100).round(2)
    validos = s.notna().sum()
    t["pct_validos"] = np.where(pd.isna(t.index), np.nan,
                                (t.n / validos * 100).round(2))
    print(f"\n=== {nombre}  (n={len(s):,}, válidos={validos:,})")
    print(t.to_string())
```

```

return t

print(f"BLOQUE A - hora-estación (n = {len(H):,})")
for c in ["temporada", "periodo", "tipo_dia", "fin_de_semana", "llovio"]:
    frecuencia(H[c], c)

```

BLOQUE A - hora-estación (n = 574,343)

```

=== temporada (n=574,343, válidos=574,343)
      n  pct_total  pct_validos

```

```

temporada
primavera  161184      28.06      28.06
invierno   147623      25.70      25.70
verano     138864      24.18      24.18
otoño      126672      22.06      22.06

```

```

=== periodo (n=574,343, válidos=574,343)
      n  pct_total  pct_validos

```

```

periodo
mañana     143586      25.0      25.0
tarde      143586      25.0      25.0
noche      143586      25.0      25.0
madrugada  143585      25.0      25.0

```

```

=== tipo_dia (n=574,343, válidos=574,343)
      n  pct_total  pct_validos

```

```

tipo_dia
laborable  390239      67.95      67.95
domingo    79728      13.88      13.88
sabado     79392      13.82      13.82
festivo    24984       4.35       4.35

```

```

=== fin_de_semana (n=574,343, válidos=574,343)
      n  pct_total  pct_validos

```

```

fin_de_semana
0           409943      71.38      71.38
1           164400      28.62      28.62

```

```

=== llovio (n=574,343, válidos=557,092)
      n  pct_total  pct_validos

```

```

llovio
0.0       549104      95.61      98.57
NaN        17251       3.00      NaN
1.0         7988       1.39      1.43

```

```
print(f"BLOQUE B - día-estación (n = {len(D):,})")
for c in ["temporada", "tipo_dia", "excede_anio_1", "excede_anio_3",
         "excede_anio_5", "excede_1h"]:
    frecuencia(D[c], c)
```

BLOQUE B - día-estación (n = 23,931)

=== temporada (n=23,931, válidos=23,931)

	n	pct_total	pct_validos
temporada			
primavera	6716	28.06	28.06
invierno	6151	25.70	25.70
verano	5786	24.18	24.18
otoño	5278	22.06	22.06

=== tipo_dia (n=23,931, válidos=23,931)

	n	pct_total	pct_validos
tipo_dia			
laborable	16260	67.95	67.95
domingo	3322	13.88	13.88
sabado	3308	13.82	13.82
festivo	1041	4.35	4.35

=== excede_anio_1 (n=23,931, válidos=22,529)

	n	pct_total	pct_validos
excede_anio_1			
0.0	20129	84.11	89.35
1.0	2400	10.03	10.65
NaN	1402	5.86	NaN

=== excede_anio_3 (n=23,931, válidos=22,529)

	n	pct_total	pct_validos
excede_anio_3			
0.0	18915	79.04	83.96
1.0	3614	15.10	16.04
NaN	1402	5.86	NaN

=== excede_anio_5 (n=23,931, válidos=22,529)

	n	pct_total	pct_validos
excede_anio_5			
0.0	15612	65.24	69.3
1.0	6917	28.90	30.7
NaN	1402	5.86	NaN

=== excede_1h (n=23,931, válidos=23,931)

	n	pct_total	pct_validos
excede_1h			

0	22019	92.01	92.01
1	1912	7.99	7.99

Verificación contra #14. `excede_anio_1` debe dar 10.65 % sobre válidos (2,400 de 22,529). Sobre el total de días-estación daría 10.03 %; la diferencia es solo la base, y por eso se reportan las dos columnas.

2. estacion cruzada con la cobertura temporal

La frecuencia absoluta de `estacion` mide tiempo de operación, no volumen de información útil. Se reporta junto con la ventana de operación y con el porcentaje de horas que efectivamente tienen O medido.

```
cov = H.groupby("estacion").agg(n=("fecha", "size"),
                               ini=("fecha", "min"), fin=("fecha", "max"))
cov["horas_teoricas"] = ((cov.fin - cov.ini).dt.total_seconds() / 3600 +
    ↪ 1).astype(int)
cov["completitud_rejilla"] = (cov.n / cov.horas_teoricas * 100).round(1)
cov["pct_filas"] = (cov.n / cov.n.sum() * 100).round(2)
cov["pct_o3_medido"] = H.groupby("estacion").03.apply(
    lambda s: round(s.notna().mean() * 100, 1))
print(cov.sort_values("n", ascending=False).to_string())
```

	n	ini	fin	horas_teoricas	completitud_rejilla
estacion					
CE	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
NE	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
NE2	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
NE3	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
NO	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
NO2	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
NTE	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
NTE2	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
SE	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
SE2	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
SE3	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
SO	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
SO2	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
SUR	39408	2021-01-01 00:00:00	2025-06-30 23:00:00	39408	100.0
NO3	22631	2022-12-01 01:00:00	2025-06-30 23:00:00	22631	100.0

3. Mediana en escala ordinal

`periodo` es la única ordinal. Se adopta el orden del ciclo diario sin cerrarlo en círculo: la mediana ordinal no tiene análogo circular limpio como sí lo tiene la codificación seno/coseno usada para `hora` y `mes` en #14. Se define como el valor en la posición $\lceil n/2 \rceil$; no se promedian categorías.

```

cod = dict(zip(ORDEN_PERIODO, range(4)))

def mediana_ordinal(s: pd.Series, etiqueta: str) -> str:
    x = np.sort(s.map(cod).dropna().values); n = len(x)
    med = ORDEN_PERIODO[x[int(np.ceil(n / 2)) - 1]]
    dist = np.round(np.bincount(x, minlength=4) / n * 100, 2)
    print(f"[{etiqueta}] n={n:,} -> mediana = {med}")
    print("    distribución%:", dict(zip(ORDEN_PERIODO, dist)))
    return med

mediana_ordinal(H.periodo, "rejilla horaria completa")
mediana_ordinal(H.loc[H.O3.notna(), "periodo"], "filas con O3 válido")

```

```

[rejilla horaria completa] n=574,343 -> mediana = tarde
    distribución %: {'madrugada': np.float64(25.0), 'mañana': np.float64(25.0), 'tarde': np.float64(25.0), 'noche': np.float64(25.0)}
[filas con O3 válido] n=549,119 -> mediana = tarde
    distribución %: {'madrugada': np.float64(24.91), 'mañana': np.float64(24.8), 'tarde': np.float64(24.9), 'noche': np.float64(25.39)}

'tarde'

```

Sobre la rejilla completa el resultado es un artefacto: cuatro bloques equifrecuentes hacen que la posición central caiga sobre la primera observación de **tarde**, a un registro de la frontera con **mañana**. Sobre las filas con O medido las frecuencias dejan de ser uniformes —los faltantes se concentran de noche— y la mediana sí queda determinada por los datos. Esa es la cifra que se reporta.

temporada se deja como nominal pura: su orden también es cíclico y no tiene un corte natural que justifique linealizarlo.

4. Visualización

Barras para todo lo que tenga más de dos niveles; pastel solo para las binarias, donde la proporción se lee sin recurrir al eje.

```

plt.rcParams.update({"font.size": 9, "font.family": "serif",
                    "axes.spines.top": False, "axes.spines.right": False,
                    "figure.dpi": 150})
GRIS, AZUL, ROJO, CLARO = "#8c8c8c", "#3b6ea5", "#b5443a", "#d9d9d9"

```

```

c = cov.sort_values("n")
col = [ROJO if e == "N03" else AZUL for e in c.index]
fig, ax = plt.subplots(1, 2, figsize=(7.5, 4), gridspec_kw={"width_ratios": [2,
↪ 1]})
ax[0].barh(c.index, c.n, color=col)
for i, v in enumerate(c.n):
    ax[0].text(v + 500, i, f"{v:,}", va="center", fontsize=7.5)
ax[0].set_xlim(0, 46000); ax[0].set_xlabel("Registros hora-estación")
ax[0].set_title("Frecuencia absoluta por estación", loc="left", fontsize=10)
ax[1].barh(c.index, c.pct_o3_medido, color=col); ax[1].set_xlim(0, 100)
ax[1].set_xlabel("% de horas con 0$_3$ medido"); ax[1].set_yticklabels([])
ax[1].set_title("Compleitud de 0$_3$", loc="left", fontsize=10)
fig.text(0.01, 0.01, "N03 (rojo) inicia operación el 2022-12-01; su menor "
        "frecuencia es cobertura temporal, no pérdida de datos.",
        fontsize=7, color="#555")
fig.tight_layout(rect=[0, 0.035, 1, 1])
fig.savefig(FIGS / "eda_cual_estacion.pdf"); plt.show()

```

/var/folders/ks/j310rx014qx5jxxp12t_qrqw0000gn/T/ipykernel_58705/2373028277.py:16: UserWarning: interactive, and thus cannot be shown

```
fig.savefig(FIGS / "eda_cual_estacion.pdf"); plt.show()
```

Figura 1

```

fig, ax = plt.subplots(figsize=(7.5, 3.2))
cv = cov.sort_index(ascending=False)
for i, (e, r) in enumerate(cv.iterrows()):
    ax.barh(i, (r.fin - r.ini).days, left=r.ini, height=.6,
            color=ROJO if e == "N03" else AZUL)
ax.set_yticks(range(len(cv))); ax.set_yticklabels(cv.index)
ax.set_title("Ventana de operación por estación (2021-01-01 a 2025-06-30)",
            loc="left", fontsize=10)
fig.tight_layout(); fig.savefig(FIGS / "eda_cual_cobertura.pdf"); plt.show()

```

/var/folders/ks/j310rx014qx5jxxp12t_qrqw0000gn/T/ipykernel_58705/3789231940.py:9: UserWarning: interactive, and thus cannot be shown

```
fig.tight_layout(); fig.savefig(FIGS / "eda_cual_cobertura.pdf"); plt.show()
```

Figura 2


```

# Tres paneles apilados y barras horizontales: la figura va a una columna del
# reporte (~3.4 in), no a doble columna. Apilada, la etiqueta de cada nivel cabe
# en el eje y sin rotar, que es lo que permite bajar el tamaño de letra.
fig, axs = plt.subplots(3, 1, figsize=(3.4, 4.3))
paneles = [(H.temporada, ["invierno", "primavera", "verano", "otoño"], "Temporada"),
            (H.periodo, ORDEN_PERIODO, "Periodo del
↪ día"),
            (H.tipo_dia, ["laborable", "sabado", "domingo", "festivo"], "Tipo de
↪ día")]
for a, (s, orden, tit) in zip(axs, paneles):
    p = s.value_counts().reindex(orden) / len(s) * 100
    y = range(len(orden))
    a.barh(y, p, color=AZUL, height=.62)
    a.set_yticks(y)
    a.set_yticklabels([o.capitalize() for o in orden], fontsize=7.5)
    a.invert_yaxis()
    for i, x in enumerate(p):
        a.text(x + max(p) * .015, i, f"{x:.1f}", va="center", fontsize=7)
    a.set_xlim(0, max(p) * 1.22); a.tick_params(axis="x", labelsz=7)
    a.set_title(tit, loc="left", fontsize=9)
    for lado in ("top", "right"):
        a.spines[lado].set_visible(False)
axs[2].set_xlabel("% de registros hora-estación", fontsize=7.5)
axs[1].axvline(25, ls="--", lw=.8, color=GRIS)
fig.tight_layout(pad=.4); fig.savefig(FIGS / "eda_cual_nominales.pdf"); plt.show()

```

/var/folders/ks/j310rx014qx5jxxp12t_qrqw0000gn/T/ipykernel_58705/3464583473.py:23: UserWarning: interactive, and thus cannot be shown

```
fig.tight_layout(pad=.4); fig.savefig(FIGS / "eda_cual_nominales.pdf"); plt.show()
```

Figura 3

```

v = [(H.llovio == 0).sum(), (H.llovio == 1).sum(), H.llovio.isna().sum()]
fig, ax = plt.subplots(figsize=(3.6, 3.6))
ax.pie(v, labels=["No llovió", "Llovió", "Sin dato"], colors=[CLARO, AZUL, GRIS],
        autopct=lambda p: f"{p:.2f}%", startangle=90, textprops={"fontsize": 8},
        wedgeprops={"edgecolor": "white", "linewidth": .8})
ax.set_title(f"llovio (n = {len(H):,} hora-estación)", fontsize=10)
fig.tight_layout(); fig.savefig(FIGS / "eda_cual_llovio.pdf"); plt.show()

```

/var/folders/ks/j310rx014qx5jxxp12t_qrqw0000gn/T/ipykernel_58705/2231852956.py:7: UserWarning: interactive, and thus cannot be shown

```
fig.tight_layout(); fig.savefig(FIGS / "eda_cual_llovio.pdf"); plt.show()
```

Figura 4

```

# Rejilla 2x2 en vez de 1x3: la figura va a una columna del reporte. Tres pays
# en fila a ~3.4 in de ancho quedarian de 1.1 in y sin espacio para los
# porcentajes; en 2x2 el diametro se conserva y la celda libre aloja la leyenda,
# que asi deja de repetirse tres veces.
fig, axs = plt.subplots(2, 2, figsize=(3.4, 3.5))
axs = axs.ravel()
for a, (c_, u) in zip(axs, [("excede_anio_1", 65), ("excede_anio_3", 60),
                           ("excede_anio_5", 51)]):
    s = D[c_]
    cunas, _, _ = a.pie([(s == 0).sum(), (s == 1).sum()],
                        colors=[CLARO, ROJO], startangle=90, autopct=lambda p: f"{p:.2f}%",
                        textprops={"fontsize": 6.5},
    ↪ wedgeprops={"edgecolor": "white", "linewidth": .8})
    a.set_title(f"{u} ppb", fontsize=8.5)
axs[3].axis("off")
axs[3].legend(cunas, ["No excede", "Excede"], loc="center", fontsize=7.5,
              frameon=False)
fig.suptitle("Balance de clases del MDA8\n"
            f"(n = {int(D.excede_anio_1.notna().sum()):,} días-estación válidos)",
            fontsize=8.5)
fig.tight_layout(pad=.3)
fig.savefig(FIGS / "eda_cual_excedencias.pdf", bbox_inches="tight"); plt.show()

```

```

/var/folders/ks/j310rx014qx5jxxp12t_qrqw0000gn/T/ipykernel_58705/2852439423.py:21: UserWarning:
interactive, and thus cannot be shown

```

```

fig.savefig(FIGS / "eda_cual_excedencias.pdf", bbox_inches="tight"); plt.show()

```

Figura 5

Al umbral vigente de 65 ppb las excedencias son el 10.65 % de los días-estación válidos: un clasificador que prediga siempre la clase mayoritaria acierta 89.35 % sin capturar nada. La exactitud queda descartada como métrica en la etapa de clasificación. El umbral de 51 ppb (30.70 %) está mucho mejor balanceado y conviene como escenario principal de modelado.

5. Criterios de aceptación

OK	Frecuencias horarias suman al total
OK	Frecuencias diarias suman al total
OK	excede_anio_1 = 10.65 % sobre válidos (obtenido 10.65 %)
OK	Ningún pastel con más de 3 sectores